
REQUIREMENTS NOT MET

None

PROBLEMS ENCOUNTERED

None Encountered

FUTURE WORK/APPLICATIONS

Finally, we wrap up this awesome semester with utilizing the DAC and DMA. Through the utilization of these two peripherals, we are able to rapidly transmit data, allowing us to modify the delivered frequency to simulate musical notes. The utilization of these peripherals could allow us to create varying frequency waveforms, potentially for wireless communications and audio creation.

Thanks for the awesome semester 😊 .

PRE-LAB EXERCISES

- i. Why might you be unable to generate a desired frequency with this method of using an interrupt? Refer to the disassembly of the interrupt service routine. Additionally, temporarily change the optimization level of your compiler to -O1. Are the results any different? Why or why not?

If the timer is running too quickly, the ISR may not be available within time to actually trigger as it is still executing commands behind the scenes, which would cause it to only update when the ISR is available, seemingly at a rate around 1 kHz.

Changing the optimization level to -O1 DOES fix the error; allowing for sine waves over 1kHz (around 1500 Hz maximum), probably by removing the ISR and instead polling, allowing for much faster updates.

- ii. Would a method of synchronous polling (i.e., a method with no interrupts) result in the same issue identified in the previous exercise? In other words, would the desired frequency not initially met now be achieved? Alter your program to check your answer, and then take a screenshot of the waveform generated, again denoting a precise frequency measurement of this waveform within the screenshot.

It likely would improve the maximum frequency considerably, causing the update rate to be limited by the processor speed rather than the ISR availability.

- iii. What is the correlation between the amount of data points used to recreate the waveform and the overall quality of the waveform?

A higher data point count means higher resolution, where we are able to visualize more individual steps along the sine wave. However, with higher quality resolution, we also have more data to process and this directly lowers our processing speed, and effectively our maximum frequencies.

PSEUDOCODE / FLOWCHARTS

Lab8_1.c

Initialize Clock @ 32MHz & DACA

Sine_table[256] = sine_table data

```
While 1{  
Wait for DAC:  
Update DAC  
}
```

```
Daca_init:  
Select AREFB  
Select Single CH0  
Enable DAC & CH0
```

Lab8_2a.c

Initialize Clock @ 32MHz & DACA & TCC0
Enable SEI

Infinite while loop{}

Daca_init from lab8_1.c

```
Tcc0_init:  
Set period to 659.26 Hz  
Set Prescaler  
Enable lo-level interrupts  
Enable PMIC for lo-level interrupts
```

```
ISR(TCC0_OVF):  
If (count < 255):  
    Count += 1  
Else:  
    Count = 0
```

DACA.DATA = sine_table[count]

Lab8_2b.c

Initialize Clock @ 32MHz & DACA & TCC0
Enable SEI

```
Infinite while loop:
    If TCC0 OVF:
        DACA.DATA = sine_table[count];
        If count < 255:
            Count += 1
        Else:
            Count = 0
```

```
Daca_init:
    Select AREFB (2.5V)
    Select CH0
    Enable DAC & CH0
```

```
Tcc0_init:
    Select period
    Select prescaler
```

Lab8_3.c

Initialize Clock @ 32MHz & DACA & TCC0 & DMAC

Infinite while loop:

```
Daca_init:
    Select AREFB (2.5V)
    Select CH0
    Enable DAC & CH0
```

```
Tcc0_init:
    Select period
    Select prescaler
```

```
Dmac_init:
    Select infinite repeats
    Address source & destination
    TRFCNT = 512 (256 * 2)

    Select REPEAT, SINGLE, 2BYTE for CTRLA
    Select BLOCK SRC LD, INC SRCDIR, BURST DESTRELOAD, INC DESTDIR for ADDRCTLR

    EVSYS fires on TCC0 OVF, DMAC triggered by EVSYS CH0

    Enable DMA / CH0
```

Lab8_4.c

Initialize Clock @ 32MHz & DACA & TCC0 & DMAC

Enable Audio Amp

Infinite while loop:

Daca_init:

Select AREFB (2.5V)

Select CH1

Enable DAC & CH0

Tcc0_init:

Reset CNT

Reset flags

Select period

Select prescaler

Dmac_init:

Select infinite repeats

Address source & destination

TRFCNT = 512 (256 * 2)

Select REPEAT, SINGLE, 2BYTE for CTRLA

Select BLOCK SRC LD, INC SRCDIR, BURST DESTRELOAD, INC DESTDIR for ADDRCTRL

EVSYS fires on TCC0 OVF, DMAC triggered by EVSYS CH0

Enable DMA / CH0

```
Lab8_5.c
Initialize Clock @ 32MHz & DACA & TCC0 & TCC1 & DMAC & USARTE0
Create baud rate, period, mode variables

// TCC0 is Playback, TCC1 is Delay

Create Sine & Tri tables
Create a Pointer, set to Sine table

Enable Audio Amp

Infinite while loop:

    Wait for USART:
        Take input character
        Compare character, set to respective note or change mode

    Wait for TCC1 OVF
        Reset flag
        Turn off playback timer


Daca_init:
    Select AREFB (2.5V)
    Select CH1
    Enable DAC & CH0

Tcc0_init:
    Reset CNT
    Reset flags
    Select period
    Select prescaler

Tcc1_init:
    Reset CNT
    Set period
    Reset flag
    Turn off


Dmac_init:
    Select infinite repeats
    Address source & destination
    TRFCNT = 512 (256 * 2)

    Select REPEAT, SINGLE, 2BYTE for CTRLA
    Select BLOCK SRC LD, INC SRCDIR, BURST DESTRELOAD, INC DESTDIR for ADDRCTRL

    EVSYS fires on TCC0 OVF, DMAC triggered by EVSYS CH0

    Enable DMA / CH0


Usartd0_init:
Set TX/RX pins
Set baud rate
Set PuTTY configs & enable

Usartd0_in:
Wait for USART:
    Return char
```

Set_wave(mode):

Disable DMA

Check mode:

 Select mode via pointer

 Update addressing

 Reset TRFCNT

Enable DMA

Set_note(freq):

 Update Tcc0.PER with freq

 Reset Tcc0.CNT

 Enable prescaler

 Reset Tcc1.CNT

 Reset Tcc1 flags

 Enable Tcc1 prescaler

Note_timer:

 Reset Tcc1.CT

 Reset Tcc1 flags

 Enable timer

PROGRAM CODE

Lab8_1.c

```
/*
 * lab8_1.c
 *
 * Created: 7/21/2026 9:25:58 PM
 * Author : Evan Baesler
 */

#include <avr/io.h>

extern void clock_init(void);

void dca_init(void);
uint16_t value = 2621;

int main(void)
{
    clock_init(); // Sets Clock to 32MHz
    dca_init();

    while (1)
    {
        // Poll until DAC channel 0 data register is empty
        if (DACA.STATUS & DAC_CH0DRE_bm)
        {
            DACA.CH0DATA = value;
        }
    }
}

void dca_init(void){
    DACA.CTRLA = DAC_REFSEL_AREFB_gc; // REFSEL[1:0] on bits 4:3, AREFB = 2.5V
    DACA.CTRLB = DAC_CHSEL_SINGLE_gc;
    DACA.CTRLA = DAC_ENABLE_bm | DAC_CH0EN_bm;
}
```

```
Lab8_2a.c
/*
 * lab8_2.c
 *
 * Created: 7/26/2026 10:15:16 PM
 * Author: Evan Baessler
 */

#include <avr/io.h>
#include <avr/interrupt.h>

extern void clock_init(void);

void dac_init(void);
void tcc0_init(void);
uint16_t count = 0;

// 256 Value Sine Wave Approximation
uint16_t sine_table[256] =
    {0x800, 0x832, 0x864, 0x896, 0x8c8, 0x8fa, 0x92c, 0x95e,
     0x98f, 0x9c0, 0x9f1, 0xa22, 0xa52, 0xa82, 0xab1, 0xae0,
     0xb0f, 0xb3d, 0xb6b, 0xb9b, 0xbc5, 0xbf1, 0xc1c, 0xc47,
     0xc71, 0xc9a, 0xcc3, 0xceb, 0xd12, 0xd39, 0xd5f, 0xd83,
     0xda7, 0xdca, 0xded, 0xe0e, 0xe2e, 0xe4e, 0xe6c, 0xe8a,
     0xea6, 0xec1, 0xedc, 0xef5, 0xf0d, 0xf24, 0xf3a, 0xf4f,
     0xf63, 0xf76, 0xf87, 0xf98, 0xfa7, 0xfb5, 0xfc2, 0fcd,
     0xfd8, 0xfe1, 0xfe9, 0xff0, 0xff5, 0xff9, 0xffd, 0xffe,
     0xffff, 0xffe, 0xffd, 0xff9, 0xff5, 0xff0, 0xfe9, 0xfe1,
     0xfd8, 0xfcd, 0xfc2, 0xfb5, 0xfa7, 0xf98, 0xf87, 0xf76,
     0xf63, 0xf4f, 0xf3a, 0xf24, 0xf0d, 0xef5, 0xedc, 0xec1,
     0xea6, 0xe8a, 0xe6c, 0xe4e, 0xe2e, 0xe0e, 0xded, 0xdca,
     0xda7, 0xd83, 0xd5f, 0xd39, 0xd12, 0xceb, 0xcc3, 0xc9a,
     0xc71, 0xc47, 0xc1c, 0xbf1, 0xbc5, 0xb9b, 0xb6b, 0xb3d,
     0xb0f, 0xae0, 0xab1, 0xa82, 0xa52, 0xa22, 0x9f1, 0x9c0,
     0x98f, 0x95e, 0x92c, 0x8fa, 0x8c8, 0x896, 0x864, 0x832,
     0x800, 0x7cd, 0x79b, 0x769, 0x737, 0x705, 0x6d3, 0x6a1,
     0x670, 0x63f, 0x60e, 0x5dd, 0x5ad, 0x57d, 0x54e, 0x51f,
     0x4f0, 0x4c2, 0x494, 0x467, 0x43a, 0x40e, 0x3e3, 0x3b8,
     0x38e, 0x365, 0x33c, 0x314, 0x2ed, 0x2c6, 0x2a0, 0x27c,
     0x258, 0x235, 0x212, 0x1f1, 0x1d1, 0x1b1, 0x193, 0x175,
     0x159, 0x13e, 0x123, 0x10a, 0xf2, 0xdb, 0xc5, 0xb0,
     0x9c, 0x89, 0x78, 0x67, 0x58, 0x4a, 0x3d, 0x32,
     0x27, 0x1e, 0x16, 0xf, 0xa, 0x6, 0x2, 0x1,
     0x0, 0x1, 0x2, 0x6, 0xa, 0xf, 0x16, 0x1e,
     0x27, 0x32, 0x3d, 0x4a, 0x58, 0x67, 0x78, 0x89,
     0x9c, 0xb0, 0xc5, 0xdb, 0xf2, 0x10a, 0x123, 0x13e,
     0x159, 0x175, 0x193, 0x1b1, 0x1d1, 0x1f1, 0x212, 0x235,
     0x258, 0x27c, 0x2a0, 0x2c6, 0x2ed, 0x314, 0x33c, 0x365,
     0x38e, 0x3b8, 0x3e3, 0x40e, 0x43a, 0x467, 0x494, 0x4c2,
     0x4f0, 0x51f, 0x54e, 0x57d, 0x5ad, 0x5dd, 0x60e, 0x63f,
     0x670, 0x6a1, 0x6d3, 0x705, 0x737, 0x769, 0x79b, 0x7cd};
```

```
int main(void)
{
    clock_init(); // Sets Clock to 32MHz
    dacc_init();
    tcc0_init();
    sei();

    while (1)
    {
        // Poll until DAC channel 0 data register is empty
    }
}

void dacc_init(void){
    DAC.CTRLA = DAC_REFSEL_AREFB_gc; // REFSEL[1:0] on bits 4:3, AREFB = 2.5V
    DAC.CTRLB = DAC_CHSEL_SINGLE_gc;
    DAC.CTRLA = DAC_ENABLE_bm | DAC_CH0EN_bm;
}

void tcc0_init(void){
    TCC0.PER = 192; // 1/659.26(32MHz/1) = 48539/256 = 190, adjusted to 192 for accuracy based on
clock speed variance
    TCC0.CTRLA = TC_CLKSEL_DIV1_gc;
    TCC0.INTCTRLA = TC_OVFINTLVL_LO_gc;
    PMIC.CTRL = PMIC_LOLVLEN_bm;
}

ISR(TCC0_OVF_vect){
    if (count < 255){
        count += 1;
    }
    else {
        count = 0;
    }

    DAC.CH0DATA = sine_table[count];
}
```

Lab8_2b.c

```
/*
 * lab8_2.c
 *
 * Created: 7/27/2026 10:58:55 PM
 * Author: Evan Baessler
 */

#include <avr/io.h>
#include <avr/interrupt.h>

extern void clock_init(void);

void daca_init(void);
void tcc0_init(void);
uint16_t count = 0;

// 256 Value Sine Wave Approximation
uint16_t sine_table[256] =
    {0x800, 0x832, 0x864, 0x896, 0x8c8, 0x8fa, 0x92c, 0x95e,
     0x98f, 0x9c0, 0x9f1, 0xa22, 0xa52, 0xa82, 0xab1, 0xae0,
     0xb0f, 0xb3d, 0xb6b, 0xb98, 0xbc5, 0xbf1, 0xc1c, 0xc47,
     0xc71, 0xc9a, 0xcc3, 0xceb, 0xd12, 0xd39, 0xd5f, 0xd83,
     0xda7, 0xdca, 0xded, 0xe0e, 0xe2e, 0xe4e, 0xe6c, 0xe8a,
     0xea6, 0xec1, 0xedc, 0xef5, 0xf0d, 0xf24, 0xf3a, 0xf4f,
     0xf63, 0xf76, 0xf87, 0xf98, 0xfa7, 0xfb5, 0xfc2, 0fcd,
     0xfd8, 0xfe1, 0xfe9, 0xff0, 0xff5, 0xff9, 0xffd, 0xffe,
     0xfff, 0xffe, 0xffd, 0xff9, 0xff5, 0xff0, 0xfe9, 0xfe1,
     0xfd8, 0xfcd, 0xfc2, 0xfb5, 0xfa7, 0xf98, 0xf87, 0xf76,
     0xf63, 0xf4f, 0xf3a, 0xf24, 0xf0d, 0xef5, 0xedc, 0xec1,
     0xea6, 0xe8a, 0xe6c, 0xe4e, 0xe2e, 0xe0e, 0xded, 0xdca,
     0xda7, 0xd83, 0xd5f, 0xd39, 0xd12, 0xceb, 0xcc3, 0xc9a,
     0xc71, 0xc47, 0xc1c, 0xbf1, 0xbc5, 0xb98, 0xb6b, 0xb3d,
     0xb0f, 0xae0, 0xab1, 0xa82, 0xa52, 0xa22, 0x9f1, 0x9c0,
     0x98f, 0x95e, 0x92c, 0x8fa, 0x8c8, 0x896, 0x864, 0x832,
     0x800, 0x7cd, 0x79b, 0x769, 0x737, 0x705, 0x6d3, 0x6a1,
     0x670, 0x63f, 0x60e, 0x5dd, 0x5ad, 0x57d, 0x54e, 0x51f,
     0x4f0, 0x4c2, 0x494, 0x467, 0x43a, 0x40e, 0x3e3, 0x3b8,
     0x38e, 0x365, 0x33c, 0x314, 0x2ed, 0x2c6, 0x2a0, 0x27c,
     0x258, 0x235, 0x212, 0x1f1, 0x1d1, 0x1b1, 0x193, 0x175,
     0x159, 0x13e, 0x123, 0x10a, 0xf2, 0xdb, 0xc5, 0xb0,
     0x9c, 0x89, 0x78, 0x67, 0x58, 0x4a, 0x3d, 0x32,
     0x27, 0x1e, 0x16, 0xf, 0xa, 0x6, 0x2, 0x1,
     0x0, 0x1, 0x2, 0x6, 0xa, 0xf, 0x16, 0x1e,
     0x27, 0x32, 0x3d, 0x4a, 0x58, 0x67, 0x78, 0x89,
     0x9c, 0xb0, 0xc5, 0xdb, 0xf2, 0x10a, 0x123, 0x13e,
     0x159, 0x175, 0x193, 0x1b1, 0x1d1, 0x1f1, 0x212, 0x235,
     0x258, 0x27c, 0x2a0, 0x2c6, 0x2ed, 0x314, 0x33c, 0x365,
     0x38e, 0x3b8, 0x3e3, 0x40e, 0x43a, 0x467, 0x494, 0x4c2,
     0x4f0, 0x51f, 0x54e, 0x57d, 0x5ad, 0x5dd, 0x60e, 0x63f,
     0x670, 0x6a1, 0x6d3, 0x705, 0x737, 0x769, 0x79b, 0x7cd};
```

```
int main(void)
{

    clock_init(); // Sets Clock to 32MHz
    daca_init();
    tcc0_init();
    sei();

    while (1)
    {
        if (TCC0.INTFLAGS & TC0_OVFIF_bm){
            TCC0.INTFLAGS = TC0_OVFIF_bm;
            DACA.CH0DATA = sine_table[count];
            if (count < 255){
                count += 1;
            }
            else {
                count = 0;
            }
        }
    }
}

void daca_init(void){
    DACA.CTRLA = DAC_REFSEL_AREFB_gc; // REFSEL[1:0] on bits 4:3, AREFB = 2.5V
    DACA.CTRLB = DAC_CHSEL_SINGLE_gc;
    DACA.CTRLA = DAC_ENABLE_bm | DAC_CH0EN_bm;
}

void tcc0_init(void){
    TCC0.PER = 85; // 1/659.26(32MHz/1) = 84, adjusted to 85 for accuracy based on clock speed
    variance
    TCC0.CTRLA = TC_CLKSEL_DIV1_gc;
}
```

Lab8_3.c

```
/*
 * lab8_3.c
 *
 * Created: 7/27/2026 11:31:17 PM
 * Author: Evan Baesler
 */
#include <avr/io.h>

extern void clock_init(void);
void daca_init(void);
void tcc0_init(void);
void dmac_init(void);

// 256 Value Sine Wave Approximation
uint16_t sine_table[256] =
{0x800, 0x832, 0x864, 0x896, 0x8c8, 0x8fa, 0x92c, 0x95e,
  0x98f, 0x9c0, 0x9f1, 0xa22, 0xa52, 0xa82, 0xab1, 0xae0,
  0xb0f, 0xb3d, 0xb6b, 0xb98, 0xbc5, 0xbf1, 0xc1c, 0xc47,
  0xc71, 0xc9a, 0xcc3, 0xceb, 0xd12, 0xd39, 0xd5f, 0xd83,
  0xda7, 0xdca, 0xded, 0xe0e, 0xe2e, 0xe4e, 0xe6c, 0xe8a,
  0xea6, 0xec1, 0xedc, 0xef5, 0xf0d, 0xf24, 0xf3a, 0xf4f,
  0xf63, 0xf76, 0xf87, 0xf98, 0xfa7, 0xfb5, 0xfc2, 0xfcd,
  0xfd8, 0xfe1, 0xfe9, 0xff0, 0xff5, 0xff9, 0xffd, 0xffe,
  0xfff, 0xffe, 0xffd, 0xff9, 0xff5, 0xff0, 0xfe9, 0xfe1,
  0xfd8, 0xfcd, 0xfc2, 0xfb5, 0xfa7, 0xf98, 0xf87, 0xf76,
  0xf63, 0xf4f, 0xf3a, 0xf24, 0xf0d, 0xef5, 0xedc, 0xec1,
  0xea6, 0xe8a, 0xe6c, 0xe4e, 0xe2e, 0xe0e, 0xded, 0xdca,
  0xda7, 0xd83, 0xd5f, 0xd39, 0xd12, 0xceb, 0xcc3, 0xc9a,
  0xc71, 0xc47, 0xc1c, 0xbf1, 0xbc5, 0xb98, 0xb6b, 0xb3d,
  0xb0f, 0xae0, 0xab1, 0xa82, 0xa52, 0xa22, 0x9f1, 0x9c0,
  0x98f, 0x95e, 0x92c, 0x8fa, 0x8c8, 0x896, 0x864, 0x832,
  0x800, 0x7cd, 0x79b, 0x769, 0x737, 0x705, 0x6d3, 0x6a1,
  0x670, 0x63f, 0x60e, 0x5dd, 0x5ad, 0x57d, 0x54e, 0x51f,
  0x4f0, 0x4c2, 0x494, 0x467, 0x43a, 0x40e, 0x3e3, 0x3b8,
  0x38e, 0x365, 0x33c, 0x314, 0x2ed, 0x2c6, 0x2a0, 0x27c,
  0x258, 0x235, 0x212, 0x1f1, 0x1d1, 0x1b1, 0x193, 0x175,
  0x159, 0x13e, 0x123, 0x10a, 0xf2, 0xdb, 0xc5, 0xb0,
  0x9c, 0x89, 0x78, 0x67, 0x58, 0x4a, 0x3d, 0x32,
  0x27, 0x1e, 0x16, 0xf, 0xa, 0x6, 0x2, 0x1,
  0x0, 0x1, 0x2, 0x6, 0xa, 0xf, 0x16, 0x1e,
  0x27, 0x32, 0x3d, 0x4a, 0x58, 0x67, 0x78, 0x89,
  0x9c, 0xb0, 0xc5, 0xdb, 0xf2, 0x10a, 0x123, 0x13e,
  0x159, 0x175, 0x193, 0x1b1, 0x1d1, 0x1f1, 0x212, 0x235,
  0x258, 0x27c, 0x2a0, 0x2c6, 0x2ed, 0x314, 0x33c, 0x365,
  0x38e, 0x3b8, 0x3e3, 0x40e, 0x43a, 0x467, 0x494, 0x4c2,
  0x4f0, 0x51f, 0x54e, 0x57d, 0x5ad, 0x5dd, 0x60e, 0x63f,
  0x670, 0x6a1, 0x6d3, 0x705, 0x737, 0x769, 0x79b, 0x7cd};
```

```
int main(void)
{
    clock_init(); // Sets Clock to 32MHz
    dacc_init();
    tcc0_init();
    dmac_init();

    while (1)
    {
        // NO OP
    }
}

void dacc_init(void){
    DACA.CTRLA = DAC_REFSEL_AREFB_gc; // REFSEL[1:0] on bits 4:3, AREFB = 2.5V
    DACA.CTRLB = DAC_CHSEL_SINGLE_gc;
    DACA.CTRLA = DAC_ENABLE_bm | DAC_CH0EN_bm;
}

void tcc0_init(void){
    TCC0.PER = 85; // 32MHz/(256*1479.98) ~= 84.5, adjusted for accuracy
    TCC0.CTRLA = TC_CLKSEL_DIV1_gc;
}

void dmac_init(void){

    DMA.CH0.REPCNT = 0; // Infinite Repeats

    // Source Addressing thru pointers
    DMA.CH0.SRCADDR0= (uint8_t)((uintptr_t) sine_table);
    DMA.CH0.SRCADDR1= (uint8_t)(((uintptr_t) sine_table) >> 8);
    DMA.CH0.SRCADDR2= (uint8_t)(((uint32_t)((uintptr_t)sine_table)) >>16);

    // Destination Addressing
    DMA.CH0.DESTADDR0= (uint8_t)((uintptr_t) &DACA.CH0DATA);
    DMA.CH0.DESTADDR1= (uint8_t)(((uintptr_t) &DACA.CH0DATA) >> 8);
    DMA.CH0.DESTADDR2= (uint8_t)(((uint32_t)((uintptr_t)&DACA.CH0DATA)) >>16);

    DMA.CH0.TRFCNT = 512; // 256 samples * 2 bytes

    // Turn on repeat, CH0, and 2BYTE burst
    DMA.CH0.CTRLA = DMA_CH_REPEAT_bm | DMA_CH_SINGLE_bm | DMA_CH_BURSTLEN_2BYTE_gc; // one 2-byte
sample per trigger

    // Reload full block, increment source/output, reload in bursts
    DMA.CH0.ADDRCTRL =
    DMA_CH_SRCRELOAD_BLOCK_gc |
    DMA_CH_SRCDIR_INC_gc |
    DMA_CH_DESTRELOAD_BURST_gc |
    DMA_CH_DESTDIR_INC_gc;

    // Timer overflow -> Event Channel 0 -> DMA Update
    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc;
    DMA.CH0.TRIGSRC = DMA_CH_TRIGSRC_EVSYS_CH0_gc;

    DMA.CH0.CTRLA |= DMA_CH_ENABLE_bm;
    DMA.CTRL = DMA_ENABLE_bm; // Activate DMA
}
```

Lab8_4.c

```
/*
 * lab8_4.c
 *
 * Created: 7/28/2026 9:56:28 PM
 * Author: Evan Baesler
 */

#include <avr/io.h>

// PROTOTYPES

extern void clock_init(void);
void daca_init(void);
void tcc0_init(void);
void dmac_init(void);

// 256 Value Sine Wave Approximation
uint16_t sine_table[256] =
{0x800, 0x832, 0x864, 0x896, 0x8c8, 0x8fa, 0x92c, 0x95e,
 0x98f, 0x9c0, 0x9f1, 0xa22, 0xa52, 0xa82, 0xab1, 0xae0,
 0xb0f, 0xb3d, 0xb6b, 0xb98, 0xbc5, 0xbf1, 0xc1c, 0xc47,
 0xc71, 0xc9a, 0xcc3, 0xceb, 0xd12, 0xd39, 0xd5f, 0xd83,
 0xda7, 0xdca, 0xded, 0xe0e, 0xe2e, 0xe4e, 0xe6c, 0xe8a,
 0xea6, 0xec1, 0xedc, 0xef5, 0xf0d, 0xf24, 0xf3a, 0xf4f,
 0xf63, 0xf76, 0xf87, 0xf98, 0xfa7, 0xfb5, 0xfc2, 0fcd,
 0xfd8, 0xfe1, 0xfe9, 0xff0, 0xff5, 0xff9, 0xffd, 0xffe,
 0xffff, 0xffe, 0xffd, 0xff9, 0xff5, 0xff0, 0xfe9, 0xfe1,
 0xfd8, 0xfcd, 0xfc2, 0xfb5, 0xfa7, 0xf98, 0xf87, 0xf76,
 0xf63, 0xf4f, 0xf3a, 0xf24, 0xf0d, 0xef5, 0xedc, 0xec1,
 0xea6, 0xe8a, 0xe6c, 0xe4e, 0xe2e, 0xe0e, 0xded, 0xdca,
 0xda7, 0xd83, 0xd5f, 0xd39, 0xd12, 0xceb, 0xcc3, 0xc9a,
 0xc71, 0xc47, 0xc1c, 0xbf1, 0xbc5, 0xb98, 0xb6b, 0xb3d,
 0xb0f, 0xae0, 0xab1, 0xa82, 0xa52, 0xa22, 0x9f1, 0x9c0,
 0x98f, 0x95e, 0x92c, 0x8fa, 0x8c8, 0x896, 0x864, 0x832,
 0x800, 0x7cd, 0x79b, 0x769, 0x737, 0x705, 0x6d3, 0x6a1,
 0x670, 0x63f, 0x60e, 0x5dd, 0x5ad, 0x57d, 0x54e, 0x51f,
 0x4f0, 0x4c2, 0x494, 0x467, 0x43a, 0x40e, 0x3e3, 0x3b8,
 0x38e, 0x365, 0x33c, 0x314, 0x2ed, 0x2c6, 0x2a0, 0x27c,
 0x258, 0x235, 0x212, 0x1f1, 0x1d1, 0x1b1, 0x193, 0x175,
 0x159, 0x13e, 0x123, 0x10a, 0xf2, 0xdb, 0xc5, 0xb0,
 0x9c, 0x89, 0x78, 0x67, 0x58, 0x4a, 0x3d, 0x32,
 0x27, 0x1e, 0x16, 0xf, 0xa, 0x6, 0x2, 0x1,
 0x0, 0x1, 0x2, 0x6, 0xa, 0xf, 0x16, 0x1e,
 0x27, 0x32, 0x3d, 0x4a, 0x58, 0x67, 0x78, 0x89,
 0x9c, 0xb0, 0xc5, 0xdb, 0xf2, 0x10a, 0x123, 0x13e,
 0x159, 0x175, 0x193, 0x1b1, 0x1d1, 0x1f1, 0x212, 0x235,
 0x258, 0x27c, 0x2a0, 0x2c6, 0x2ed, 0x314, 0x33c, 0x365,
 0x38e, 0x3b8, 0x3e3, 0x40e, 0x43a, 0x467, 0x494, 0x4c2,
 0x4f0, 0x51f, 0x54e, 0x57d, 0x5ad, 0x5dd, 0x60e, 0x63f,
 0x670, 0x6a1, 0x6d3, 0x705, 0x737, 0x769, 0x79b, 0x7cd};

int main(void)
{

    clock_init(); // Sets Clock to 32MHz

    daca_init(); // Digital to Analog Converter
    tcc0_init(); // Frequency Timer
    dmac_init(); // Initialize Table in Memory
```

```
// Enable audio amplifier (disable shutdown)
PORTC.DIRSET = PIN7_bm;
PORTC.OUTSET = PIN7_bm;

while (1)
{
    // NO OP
}

void daca_init(void)
{
    // Enable DAC Channel 1 for Speaker
    DACA.CTRLB = DAC_CHSEL_SINGLE1_gc;

    // Use AREFB (2.5V) as the DAC reference
    DACA.CTRLA = DAC_REFSEL_AREFB_gc;

    DACA.CTRLA = DAC_ENABLE_bm | DAC_CH1EN_bm | DAC_CH0EN_bm;
}

void tcc0_init(void)
{
    TCC0.CNT = 0;
    TCC0.INTFLAGS = TC0_OVFIF_bm;

    // 32 MHz / ((83 + 1) * 256) = 1488.1 Hz
    // Closest integer timer period to 1479.98 Hz
    TCC0.PER = 83;
    TCC0.CTRLA = TC_CLKSEL_DIV1_gc;
}
```

```
void dmac_init(void)
{
    DMA.CTRL |= DMA_RESET_bm; // Reset DMA

    DMA.CH0.TRFCNT = 512;
    DMA.CH0.REPCNT = 0;

    // Reload full block, increment source/output, reload in bursts
    DMA.CH0.ADDRCTRL =
    DMA_CH_SRCRELOAD_BLOCK_gc |
    DMA_CH_SRCDIR_INC_gc |
    DMA_CH_DESTRELOAD_BURST_gc |
    DMA_CH_DESTDIR_INC_gc;

    // Turn on Repeat, CH0, and 2BYTE burst
    DMA.CH0.CTRLA =
    DMA_CH_REPEAT_bm |
    DMA_CH_SINGLE_bm |
    DMA_CH_BURSTLEN_2BYTE_gc;

    // Timer overflow -> Event Channel 0 -> DMA Update
    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc;
    DMA.CH0.TRIGSRC = DMA_CH_TRIGSRC_EVSYS_CH0_gc;

    // Source Addressing thru pointers
    DMA.CH0.SRCADDR0= (uint8_t)((uintptr_t) sine_table);
    DMA.CH0.SRCADDR1= (uint8_t)(((uintptr_t) sine_table) >> 8);
    DMA.CH0.SRCADDR2= (uint8_t)(((uint32_t)((uintptr_t)sine_table)) >>16);

    // Destination Addressing
    DMA.CH0.DESTADDR0= (uint8_t)((uintptr_t) &DACA.CH1DATA);
    DMA.CH0.DESTADDR1= (uint8_t)(((uintptr_t) &DACA.CH1DATA) >> 8);
    DMA.CH0.DESTADDR2= (uint8_t)(((uint32_t)((uintptr_t)&DACA.CH1DATA)) >>16);

    DMA.CH0.CTRLA |= DMA_CH_ENABLE_bm;

    DMA.CTRL = DMA_ENABLE_bm;
}
```

Lab8_5.c

```
/*
 * lab8_5.c
 *
 * Created: 7/29/2026 8:53:41 AM
 * Author: Evan Baesler
 */

#include <avr/io.h>

// PROTOTYPES

extern void clock_init(void);
void usartd0_init(void);
void daca_init(void);
void tcc0_init(void);
void dmac_init(void);

char usartd0_in(void);
void set_note(float freq);
void set_wave(uint8_t mode);

uint16_t per; // Played Note Frequency
uint8_t mode = 0; // 0 = sine, 1 = triangle

// Baud Rate (9600bps from previous labs)
uint8_t BSEL = 207;
uint8_t BSCALE = 0;

// 256 Value Sine Wave Approximation
uint16_t sine_table[256] =
{0x800, 0x832, 0x864, 0x896, 0x8c8, 0x8fa, 0x92c, 0x95e,
 0x98f, 0x9c0, 0x9f1, 0xa22, 0xa52, 0xa82, 0xab1, 0xae0,
 0xb0f, 0xb3d, 0xb6b, 0xb98, 0xbc5, 0xbf1, 0xc1c, 0xc47,
 0xc71, 0xc9a, 0xcc3, 0xceb, 0xd12, 0xd39, 0xd5f, 0xd83,
 0xda7, 0xdca, 0xded, 0xe0e, 0xe2e, 0xe4e, 0xe6c, 0xe8a,
 0xea6, 0xec1, 0xedc, 0xef5, 0xf0d, 0xf24, 0xf3a, 0xf4f,
 0xf63, 0xf76, 0xf87, 0xf98, 0xfa7, 0xfb5, 0xfc2, 0xcd,
 0xfd8, 0xfe1, 0xfe9, 0xff0, 0xff5, 0xff9, 0xffd, 0xffe,
 0xfff, 0xffe, 0xffd, 0xff9, 0xff5, 0xff0, 0xfe9, 0xfe1,
 0xfd8, 0xcd, 0xfc2, 0xfb5, 0xfa7, 0xf98, 0xf87, 0xf76,
 0xf63, 0xf4f, 0xf3a, 0xf24, 0xf0d, 0xef5, 0xedc, 0xec1,
 0xea6, 0xe8a, 0xe6c, 0xe4e, 0xe2e, 0xe0e, 0xded, 0xdca,
 0xda7, 0xd83, 0xd5f, 0xd39, 0xd12, 0xceb, 0xcc3, 0xc9a,
 0xc71, 0xc47, 0xc1c, 0xbf1, 0xbc5, 0xb98, 0xb6b, 0xb3d,
 0xb0f, 0xae0, 0xab1, 0xa82, 0xa52, 0xa22, 0x9f1, 0x9c0,
 0x98f, 0x95e, 0x92c, 0x8fa, 0x8c8, 0x896, 0x864, 0x832,
 0x800, 0x7cd, 0x79b, 0x769, 0x737, 0x705, 0x6d3, 0x6a1,
 0x670, 0x63f, 0x60e, 0x5dd, 0x5ad, 0x57d, 0x54e, 0x51f,
 0x4f0, 0x4c2, 0x494, 0x467, 0x43a, 0x40e, 0x3e3, 0x3b8,
 0x38e, 0x365, 0x33c, 0x314, 0x2ed, 0x2c6, 0x2a0, 0x27c,
 0x258, 0x235, 0x212, 0x1f1, 0x1d1, 0x1b1, 0x193, 0x175,
 0x159, 0x13e, 0x123, 0x10a, 0xf2, 0xdb, 0xc5, 0xb0,
 0x9c, 0x89, 0x78, 0x67, 0x58, 0x4a, 0x3d, 0x32,
 0x27, 0x1e, 0x16, 0xf, 0xa, 0x6, 0x2, 0x1,
 0x0, 0x1, 0x2, 0x6, 0xa, 0xf, 0x16, 0x1e,
 0x27, 0x32, 0x3d, 0x4a, 0x58, 0x67, 0x78, 0x89,
 0x9c, 0xb0, 0xc5, 0xdb, 0xf2, 0x10a, 0x123, 0x13e,
 0x159, 0x175, 0x193, 0x1b1, 0x1d1, 0x1f1, 0x212, 0x235,
 0x258, 0x27c, 0x2a0, 0x2c6, 0x2ed, 0x314, 0x33c, 0x365,
 0x38e, 0x3b8, 0x3e3, 0x40e, 0x43a, 0x467, 0x494, 0x4c2,
```

```
0x4f0, 0x51f, 0x54e, 0x57d, 0x5ad, 0x5dd, 0x60e, 0x63f,  
0x670, 0x6a1, 0x6d3, 0x705, 0x737, 0x769, 0x79b, 0x7cd};
```

```
// 256 Value Triangle Wave Approximation
```

```
uint16_t tri_table[256] =  
{0x00, 0x20, 0x40, 0x60, 0x80, 0xa0, 0xc0, 0xe0, 0x100,  
0x120, 0x140, 0x160, 0x180, 0x1a0, 0x1c0, 0x1e0, 0x200,  
0x220, 0x240, 0x260, 0x280, 0x2a0, 0x2c0, 0x2e0, 0x300,  
0x320, 0x340, 0x360, 0x380, 0x3a0, 0x3c0, 0x3e0, 0x400,  
0x420, 0x440, 0x460, 0x480, 0x4a0, 0x4c0, 0x4e0, 0x500,  
0x520, 0x540, 0x560, 0x580, 0x5a0, 0x5c0, 0x5e0, 0x600,  
0x620, 0x640, 0x660, 0x680, 0x6a0, 0x6c0, 0x6e0, 0x700,  
0x720, 0x740, 0x760, 0x780, 0x7a0, 0x7c0, 0x7e0, 0x800,  
0x81f, 0x83f, 0x85f, 0x87f, 0x89f, 0x8bf, 0x8df, 0x8ff,  
0x91f, 0x93f, 0x95f, 0x97f, 0x99f, 0x9bf, 0x9df, 0x9ff,  
0xa1f, 0xa3f, 0xa5f, 0xa7f, 0xa9f, 0xabf, 0xadf, 0xaff,  
0xb1f, 0xb3f, 0xb5f, 0xb7f, 0xb9f, 0xbbf, 0xbdf, 0xbff,  
0xc1f, 0xc3f, 0xc5f, 0xc7f, 0xc9f, 0xcbf, 0xcdf, 0xcff,  
0xd1f, 0xd3f, 0xd5f, 0xd7f, 0xd9f, 0xdbf, 0ddf, 0xdf,  
0xe1f, 0xe3f, 0xe5f, 0xe7f, 0xe9f, 0xebf, 0xedf, 0xef,  
0xf1f, 0xf3f, 0xf5f, 0xf7f, 0xf9f, 0xfb, 0xfd, 0xff,  
0xfdf, 0xfbf, 0xf9f, 0xf7f, 0xf5f, 0xf3f, 0xf1f, 0xef,  
0xedf, 0xebf, 0xe9f, 0xe7f, 0xe5f, 0xe3f, 0xe1f, 0xdf,  
0xddf, 0xdbf, 0xd9f, 0xd7f, 0xd5f, 0xd3f, 0xd1f, 0xcff,  
0xcdf, 0xcbf, 0xc9f, 0xc7f, 0xc5f, 0xc3f, 0xc1f, 0xbff,  
0xbdf, 0xbbf, 0xb9f, 0xb7f, 0xb5f, 0xb3f, 0xb1f, 0xaff,  
0xadf, 0xabf, 0xa9f, 0xa7f, 0xa5f, 0xa3f, 0xa1f, 0x9ff,  
0x9df, 0x9bf, 0x99f, 0x97f, 0x95f, 0x93f, 0x91f, 0x8ff,  
0x8df, 0x8bf, 0x89f, 0x87f, 0x85f, 0x83f, 0x81f, 0x800,  
0x7e0, 0x7c0, 0x7a0, 0x780, 0x760, 0x740, 0x720, 0x700,  
0x6e0, 0x6c0, 0x6a0, 0x680, 0x660, 0x640, 0x620, 0x600,  
0x5e0, 0x5c0, 0x5a0, 0x580, 0x560, 0x540, 0x520, 0x500,  
0x4e0, 0x4c0, 0x4a0, 0x480, 0x460, 0x440, 0x420, 0x400,  
0x3e0, 0x3c0, 0x3a0, 0x380, 0x360, 0x340, 0x320, 0x300,  
0x2e0, 0x2c0, 0x2a0, 0x280, 0x260, 0x240, 0x220, 0x200,  
0x1e0, 0x1c0, 0x1a0, 0x180, 0x160, 0x140, 0x120, 0x100,  
0xe0, 0xc0, 0xa0, 0x80, 0x60, 0x40, 0x20};
```

```
// Point to a Table
```

```
uint16_t *waveform_ptr = sine_table;
```

```
int main(void)  
{
```

```
    clock_init(); // Sets Clock to 32MHz
```

```
    usartd0_init(); // Initialize USART
```

```
    dac_init(); // Digital To Analog Converter
```

```
    tcc0_init(); // Audio Playback Timer
```

```
    tcc1_init(); // Audio Delay Timer
```

```
    dmac_init(); // Initialize table in memory
```

```
    // Enable audio amplifier (disable shutdown)
```

```
    PORTC.DIRSET = PIN7_bm;
```

```
    PORTC.OUTSET = PIN7_bm;
```

```
    while (1)
```

```
    {
```

```
        if(USARTD0.STATUS & USART_RXCIF_bm) // Wait for USART to be ready
```

```
        {
```

```
            char c = usartd0_in(); // Get input character
```

```
        if(c=='s' || c=='S') // Do both 's' or 'S' for caps insensitivity
        {
            mode ^= 1; // Flip between 0/1
            set_wave(mode); // Update mode
        }
        else if(c=='e' || c=='E')
            set_note(1046.50); // Play C6
        else if(c=='4')
            set_note(1108.73); // Play C#6 / Db6
        else if(c=='r' || c=='R')
            set_note(1174.66); // Play D6
        else if(c=='5')
            set_note(1244.51); // Play D#6 / Eb6
        else if(c=='t' || c=='T')
            set_note(1318.51); // Play E6
        else if(c=='y' || c=='Y')
            set_note(1396.91); // Play F6
        else if(c=='7')
            set_note(1479.98); // Play F#6 / Gb6
        else if(c=='u' || c=='U')
            set_note(1567.98); // Play G6
        else if(c=='8')
            set_note(1661.22); // Play G#6 / Ab6
        else if(c=='i' || c=='I')
            set_note(1760.00); // Play A6
        else if(c=='9')
            set_note(1864.66); // Play A#6 / Bb6
        else if(c=='o' || c=='O')
            set_note(1975.53); // Play B6
        else if(c=='p' || c=='P')
            set_note(2093.00); // Play C7
    }

    if(TCC1.INTFLAGS & TC1_OVFIF_bm) // Wait for delay to end
    {
        TCC1.INTFLAGS = TC1_OVFIF_bm; // Reset flag
        TCC0.CTRLA = TC_CLKSEL_OFF_gc; // Turn off audio playback timer
    }
}

void daca_init(void)
{
    // Enable DAC Channel 1 for Speaker
    DACA.CTRLB = DAC_CHSEL_SINGLE1_gc; // Select DAC Channel 1 (PA3) (Speaker)

    // Use AREFB (2.5V) as the DAC reference
    DACA.CTRLA = DAC_REFSEL_AREFB_gc;

    DACA.CTRLA = DAC_ENABLE_bm | DAC_CH1EN_bm; // Enable CH1 DAC
}

void tcc0_init(void)
{
    // Audio Playback Timer
    TCC0.CNT = 0; // Reset TCC0 count
    TCC0.INTFLAGS = TC0_OVFIF_bm; // Reset TCC0 flags
    TCC0.CTRLA = TC_CLKSEL_OFF_gc; // Turn off playback
}
```

```
void tcc1_init(void)
{
    // Audio Delay Timer
    TCC1.CNT = 0;

    // 275ms delay (just enough to prevent skipping when holding key)
    TCC1.PER = (275e-3)*(32e6/256); // 256 PRE set in DMA
    TCC1.INTFLAGS = TC1_OVFIF_bm; // Reset flags

    TCC1.CTRLA = TC_CLKSEL_OFF_gc; // Turn off delay
}

void dmac_init(void)
{
    DMA.CTRL |= DMA_RESET_bm; // Reset DMA

    DMA.CH0.TRFCNT = 512; // 256 Data Entries * 2
    DMA.CH0.REPCNT = 0; // Infinite Repeats

    // Reload full block, increment source/output, reload in bursts
    DMA.CH0.ADDRCTRL =
        DMA_CH_SRCRELOAD_BLOCK_gc |
        DMA_CH_SRCDIR_INC_gc |
        DMA_CH_DESTDIR_INC_gc |
        DMA_CH_DESTRELOAD_BURST_gc;

    // Turn on repeat, CH0, and 2BYTE burst
    DMA.CH0.CTRLA =
        DMA_CH_REPEAT_bm |
        DMA_CH_SINGLE_bm |
        DMA_CH_BURSTLEN_2BYTE_gc;

    // Timer overflow -> Event Channel 0 -> DMA Update
    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc;
    DMA.CH0.TRIGSRC = DMA_CH_TRIGSRC_EVSYS_CH0_gc;

    // Source addressing thru pointers
    DMA.CH0.SRCADDR0= (uint8_t)((uintptr_t) sine_table);
    DMA.CH0.SRCADDR1= (uint8_t)(((uintptr_t) sine_table) >> 8);
    DMA.CH0.SRCADDR2= (uint8_t)(((uint32_t)((uintptr_t)sine_table)) >>16);

    // Destination Addressing
    DMA.CH0.DESTADDR0= (uint8_t)((uintptr_t) &DACA.CH1DATA);
    DMA.CH0.DESTADDR1= (uint8_t)(((uintptr_t) &DACA.CH1DATA) >> 8);
    DMA.CH0.DESTADDR2= (uint8_t)(((uint32_t)((uintptr_t)&DACA.CH1DATA)) >>16);

    DMA.CH0.CTRLA |= DMA_CH_ENABLE_bm;

    DMA.CTRL = DMA_ENABLE_bm;
}

void usartd0_init(void)
{
    // TX (PD3) output
    PORTD.OUTSET = PIN3_bm;
    PORTD.DIRSET = PIN3_bm;

    // RX (PD2) input
    PORTD.DIRCLR = PIN2_bm;

    // 9600 baud @ 32 MHz
```

```
    USARTD0.BAUDCTRLA = BSEL;
    USARTD0.BAUDCTRLB = BSCALE;

    // 8 data bits, no parity, 1 stop bit
    USARTD0.CTRLA = USART_CHSIZE_8BIT_gc;

    // Enable transmitter / receiver
    USARTD0.CTRLB = USART_TXEN_bm | USART_RXEN_bm;
}

char usartd0_in(void)
{
    while (!(USARTD0.STATUS & USART_RXCIF_bm)); // Wait for USART

    return USARTD0.DATA; // Sends back char from USARTD0 input
}

void set_note(float freq)
{
    per = (32000000.0 / (256.0 * freq)) - 1; // PER = SCF/(PRE*FREQ) - 1

    TCC0.PER = per; // Update frequency
    TCC0.CNT = 0; // Reset count
    TCC0.CTRLA = TC_CLKSEL_DIV1_gc; // Start audio playback timer

    TCC1.CNT = 0;
    TCC1.INTFLAGS = TC1_OVFIF_bm;
    TCC1.CTRLA = TC_CLKSEL_DIV256_gc; // Start 275ms delay timer
}

void set_wave(uint8_t mode)
{
    // Temporarily disable DMA channel during config
    DMA.CH0.CTRLA &= ~DMA_CH_ENABLE_bm;

    if(mode == 0) // Mode 0 = Sine
    {
        waveform_ptr = sine_table;
    }
    else // Mode 1 = Tri
    {
        waveform_ptr = tri_table;
    }

    // Update source address
    DMA.CH0.SRCADDR0 = (uint8_t)((uintptr_t)waveform_ptr);
    DMA.CH0.SRCADDR1 = (uint8_t)(((uintptr_t)waveform_ptr) >> 8);
    DMA.CH0.SRCADDR2 = (uint8_t)(((uint32_t)((uintptr_t)waveform_ptr)) >> 16);

    // Reset transfer count
    DMA.CH0.TRFCNT = 512;

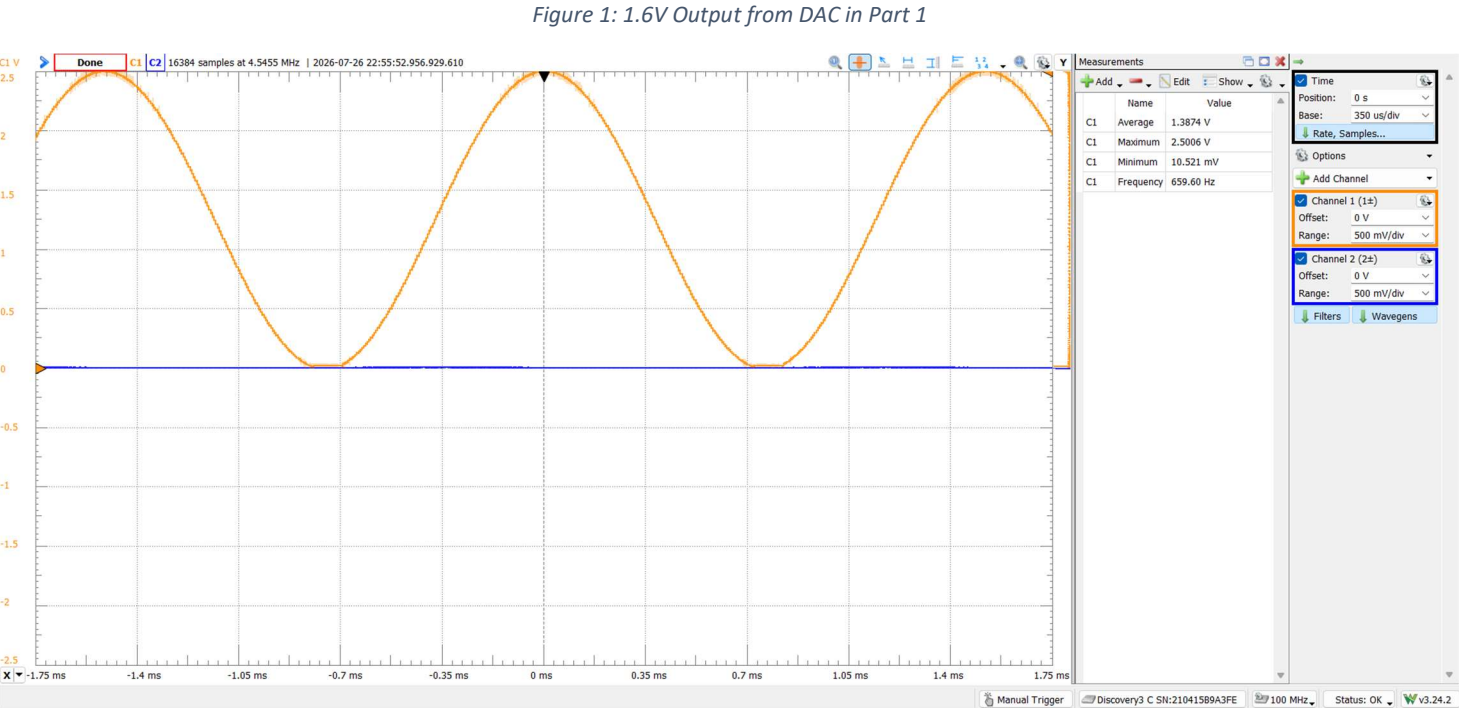
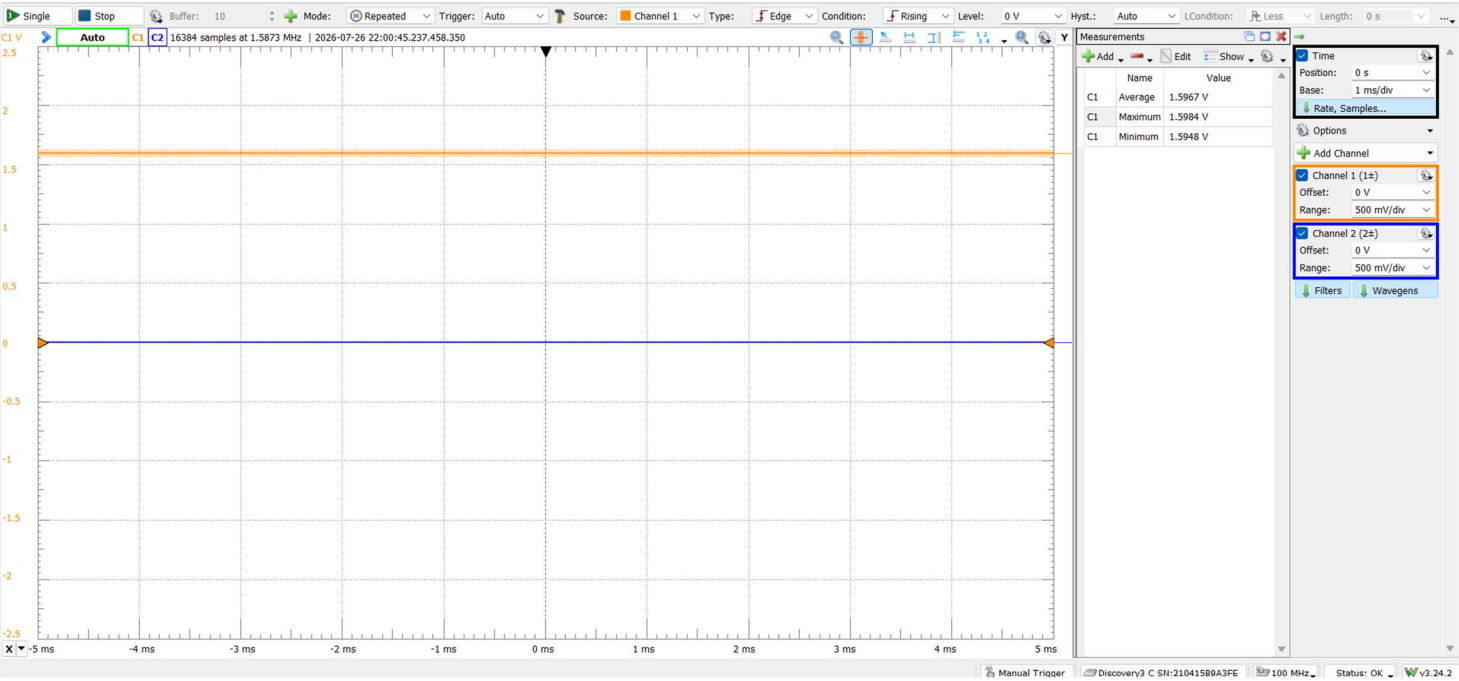
    // Re-enable DMA CH0
    DMA.CH0.CTRLA |= DMA_CH_ENABLE_bm;
}

void note_timer(void)
{
    TCC1.CNT = 0; // Reset timer

    TCC1.INTFLAGS = TC1_OVFIF_bm; // Reset flags
}
```

```
    TCC1.CTRLA = TC_CLKSEL_DIV256_gc; // Enable 275ms timer  
}
```

APPENDIX



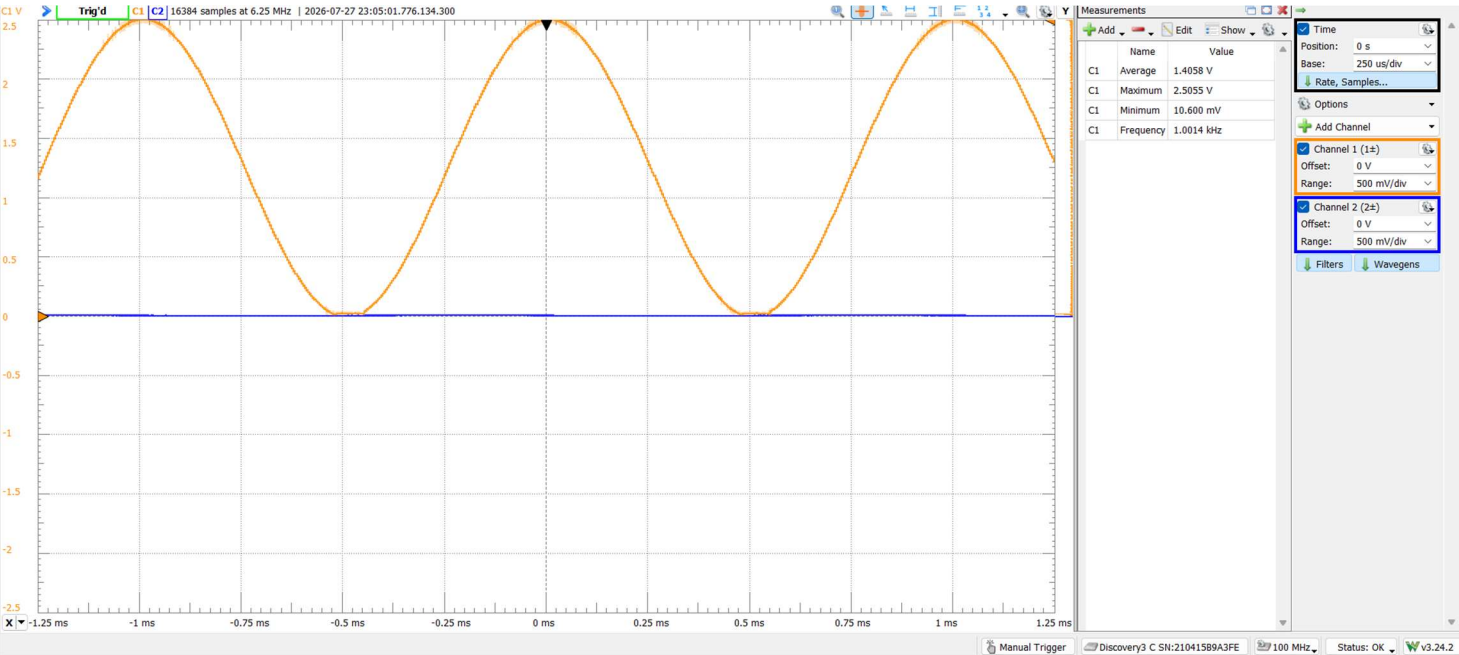


Figure 3: Attempt at 1059.26 Hz (WITH ISR), fails at 1kHz

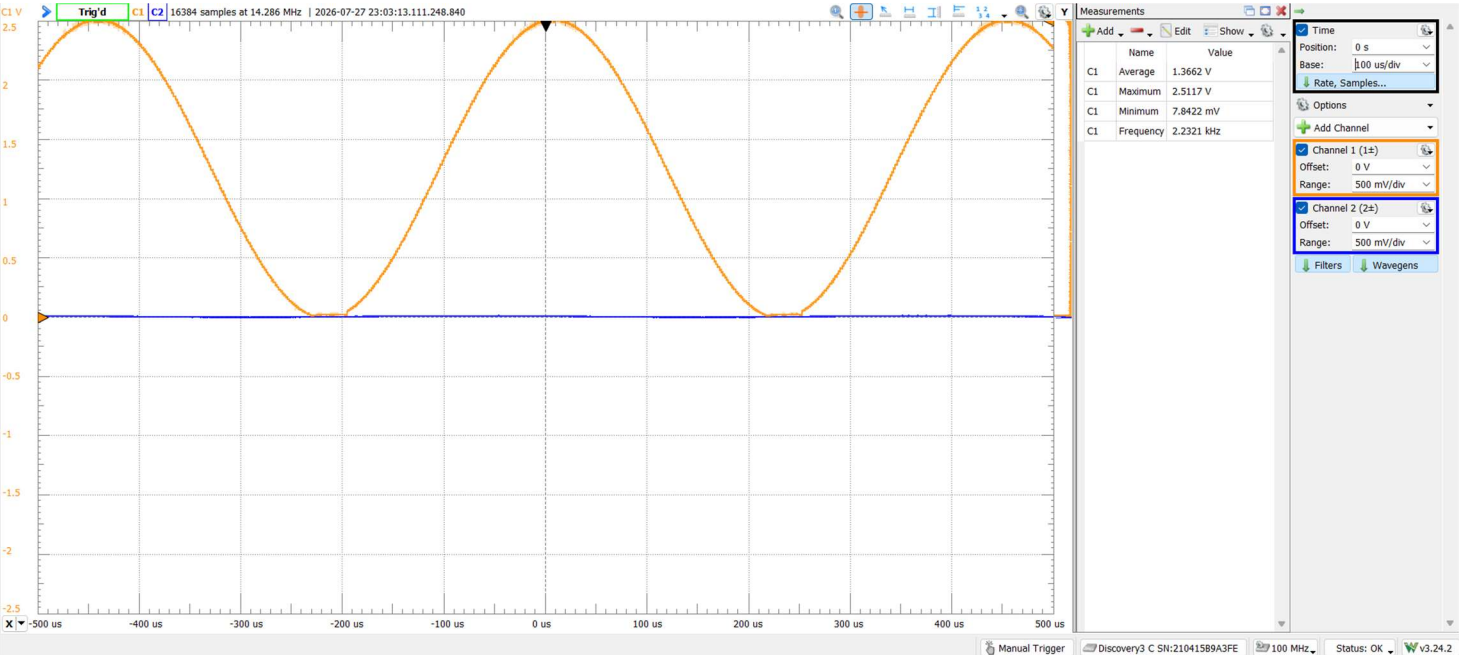


Figure 4: Max Frequency Sine Wave in lab8_2.2a (NO ISR) of 2.23kHz.

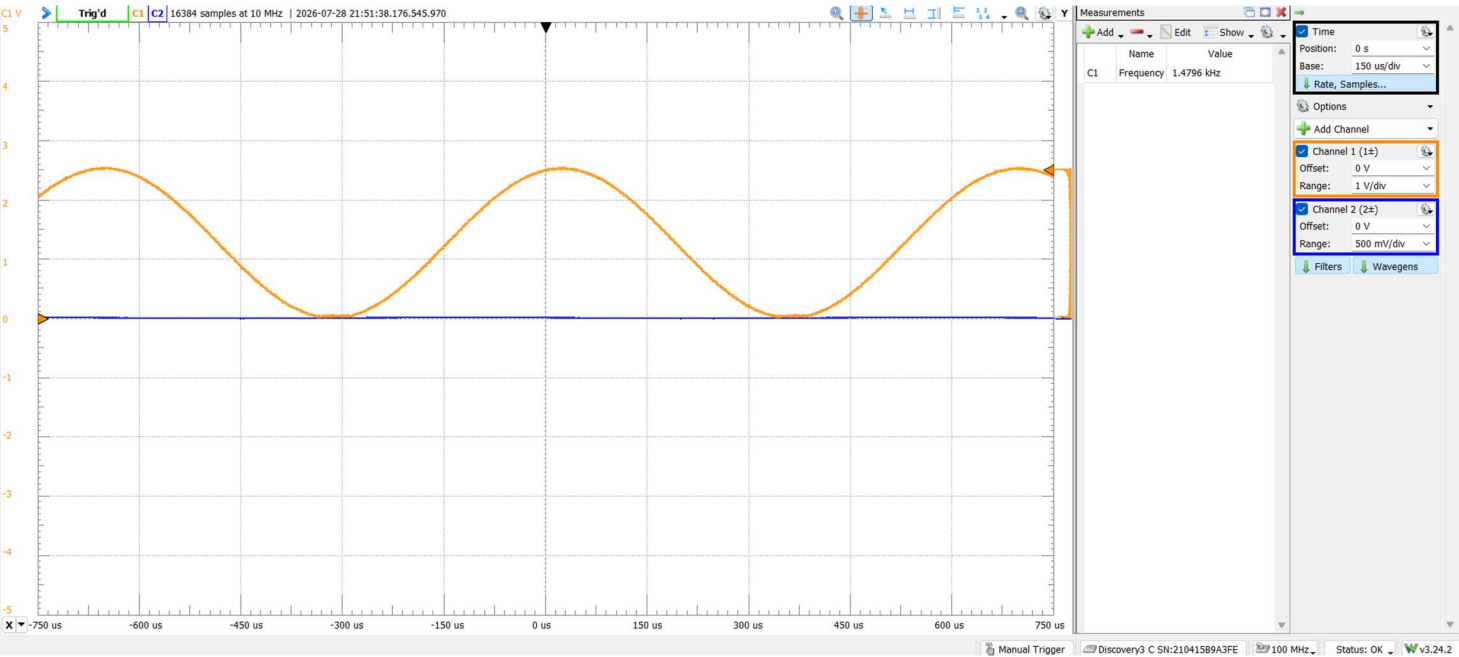


Figure 5: Lab8_2b Waveform of F# (6) or 1479.98 Hz WITHOUT DMA

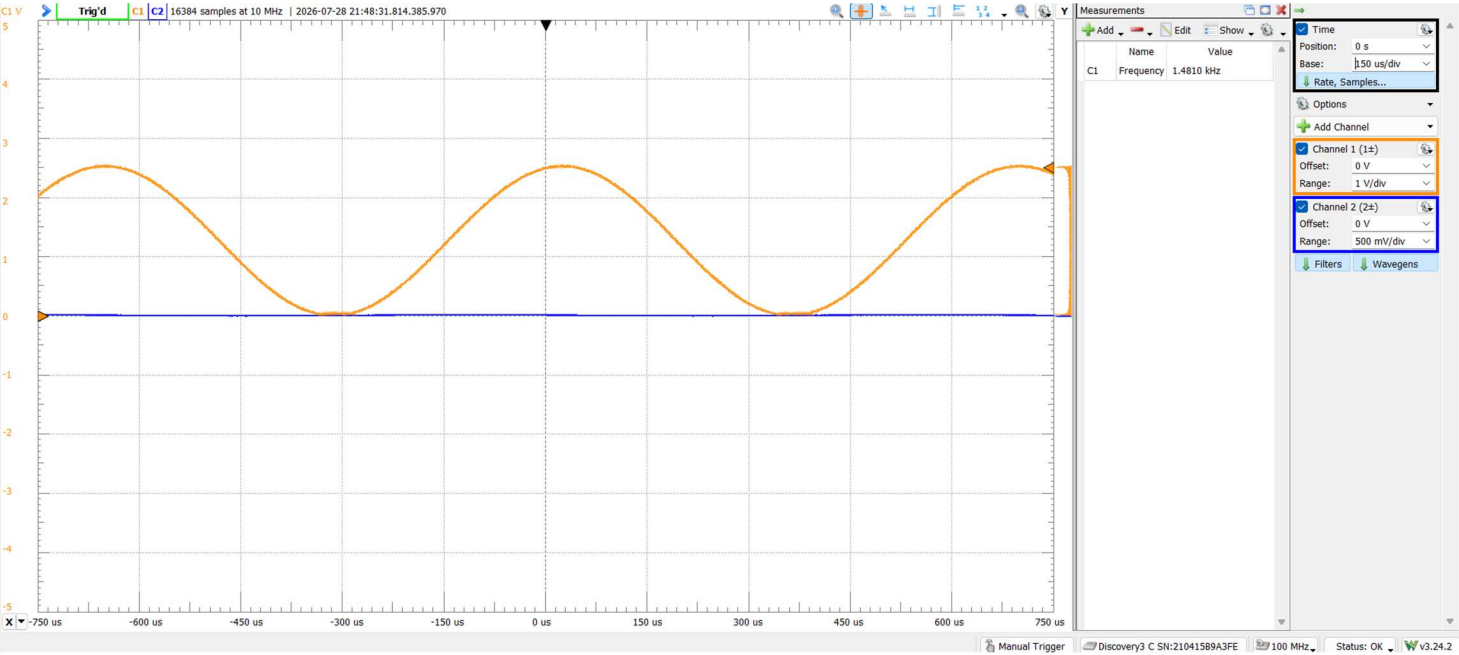


Figure 6: Lab6_3 Waveform of F# (6) or 1479.98 Hz WITH DMA

Word Address		
ATxmega128A1U		ATxmega64A1U
0		0
FFFF	/	77FF
F000	/	7800
FFFF	/	7FFF
10000	/	8000
10FFF	/	87FF

Figure 7: Program Memory Addressing

Byte Address	ATxmega64A1U	Byte Address	ATxmega128A1U
0	I/O Registers (4K)	0	I/O Registers (4KB)
FFF		FFF	
1000	EEPROM (2K)	1000	EEPROM (2K)
17FF		17FF	
	RESERVED		RESERVED
2000	Internal SRAM (4K)	2000	Internal SRAM (8K)
2FFF		3FFF	
4000	External Memory (0 to 16MB)	3000	External Memory (0 to 16MB)
FFFFFF		FFFFFF	

Figure 8: Data Memory Addressing

13.16 Register summary – Ports

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DIR	DIR[7:0]								148
+0x01	DIRSET	DIRSET[7:0]								148
+0x02	DIRCLR	DIRCLR[7:0]								148
+0x03	DIRTGL	DIRTGL[7:0]								148
+0x04	OUT	OUT[7:0]								149
+0x05	OUTSET	OUTSET[7:0]								149
+0x06	OUTCLR	OUTCLR[7:0]								149
+0x07	OUTTGL	OUTTGL[7:0]								149
+0x08	IN	IN[7:0]								150
+0x09	INTCTRL	–	–	–	–	INT1LVL[1:0]		INT0LVL[1:0]		150
+0x0A	INT0MASK	INT0MSK[7:0]								150
+0x0B	INT1MASK	INT1MSK[7:0]								150
+0x0C	INTFLAGS	–	–	–	–	–	–	INT1IF	INT0IF	151
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	REMAP	–	–	SPI	USART0	TC0D	TC0C	TC0B	TC0A	151
+0x0F	Reserved	–	–	–	–	–	–	–	–	
+0x10	PIN0CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x11	PIN1CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x12	PIN2CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x13	PIN3CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x14	PIN4CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x15	PIN5CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x16	PIN6CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x17	PIN7CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x18	Reserved	–	–	–	–	–	–	–	–	
+0x19	Reserved	–	–	–	–	–	–	–	–	
+0x1A	Reserved	–	–	–	–	–	–	–	–	
+0x1B	Reserved	–	–	–	–	–	–	–	–	
+0x1C	Reserved	–	–	–	–	–	–	–	–	
+0x1D	Reserved	–	–	–	–	–	–	–	–	
+0x1E	Reserved	–	–	–	–	–	–	–	–	
+0x1F	Reserved	–	–	–	–	–	–	–	–	

Figure 9: Ports Register Summary

14.13 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA	–	–	–	–	CLKSEL[3:0]				175
+0x01	CTRLB	CCDEN	CCCEN	CCBEN	CCAEN	–	WGMODE[2:0]			175
+0x02	CTRLC	–	–	–	–	CMPD	CMPC	CMPB	CMPA	176
+0x03	CTRLD	EVACT[2:0]			EVDLY	EVSEL[3:0]				176
+0x04	CTRLF	–	–	–	–	–	–	BYTEM		178
+0x05	Reserved	–	–	–	–	–	–	–	–	
+0x06	INTCTRLA	–	–	–	–	ERRINTLV[1:0]		OVINTLV[1:0]		178
+0x07	INTCTRLB	CCCINTLV[1:0]		CCBINTLV[1:0]		CCBINTLV[1:0]		CCAINTLV[1:0]		178
+0x08	CTRLFCLR	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	179
+0x09	CTRLFSET	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	180
+0x0A	CTRLGCLR	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0B	CTRLGSET	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0C	INTFLAGS	CCDIF	CCCIF	CCBIF	CCAIF	–	–	ERRIF	OVFIF	180
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	Reserved	–	–	–	–	–	–	–	–	
+0x0F	TEMP	TEMP[7:0]								181
+0x10 to +0x1F	Reserved	–	–	–	–	–	–	–	–	
+0x20	CNTL	CNT[7:0]								181
+0x21	CNTH	CNT[15:8]								181
+0x22 to +0x25	Reserved	–	–	–	–	–	–	–	–	
+0x26	PERL	PER[7:0]								182
+0x27	PERH	PER[8:15]								182
+0x28	CCAL	CCA[7:0]								182
+0x29	CAAH	CCA[15:8]								182
+0x2A	CCBL	CCB[7:0]								182
+0x2B	CCBH	CCB[15:8]								182
+0x2C	CCCL	CCC[7:0]								182
+0x2D	CCCH	CCC[15:8]								182
+0x2E	CCDL	CCD[7:0]								182
+0x2F	CCDH	CCD[15:8]								182
+0x30 to +0x35	Reserved	–	–	–	–	–	–	–	–	
+0x36	PERBUFL	PERBUF[7:0]								183

Figure 10: Timer/Counter Register Summary

12.9 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	STATUS	NMIEX	–	–	–	–	HILVLEX	MEDLVLEX	LOLVLEX	136
+0x01	INTPRI	INTPRI[7:0]								136
+0x02	CTRL	RREN	IVSEL	–	–	–	HILVLEN	MEDLVLEN	LOLVLEN	137

Figure 11: Interrupts Register Summary

27.12 Register summary – EBI

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRL	SDDATAW[1:0]		LPCMODE[1:0]		SRMODE[1:0]		IFMODE[1:0]		329
+0x01	SDRAMCTRLA	–	–	–	–	SDCAS	SDROW	SDCOL[1:0]		330
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	Reserved	–	–	–	–	–	–	–	–	
+0x04	REFRESHL	SDRAM Refresh Period Low Byte								331
+0x05	REFRESHH	–	–	–	–	–	–	SDRAM Refresh Period High		331
+0x06	INITDLYL	SDRAM Initialization Time Low Byte								331
+0x07	INITDLYH	–	–	SDRAM Initialization Time High Byte						331
+0x08	SDRAMCTRLB	MRDLY[1:0]		ROWCYCDLY[[2:0]			RPDLY[2:0]			332
+0x09	SDRAMCTRLC	WRDLY[1:0]		ESRDLY[2:0]			ROWCOLDLY[2:0]			333
+0x0A	Reserved	–	–	–	–	–	–	–	–	
+0x0B	Reserved	–	–	–	–	–	–	–	–	
+0x0C	Reserved	–	–	–	–	–	–	–	–	
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	Reserved	–	–	–	–	–	–	–	–	
+0x0F	Reserved	–	–	–	–	–	–	–	–	
+0x10	CS0	Chip Select 0 Offset Address								
+0x14	CS1	Chip Select 1 Offset Address								
+0x18	CS2	Chip Select 2 Offset Address								
+0x1C	CS3	Chip Select 3 Offset Address								

Figure 12: EBI Register Summary

27.13 Register summary – EBI chip select

Address	Name		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA		–	ASIZE[4:0]					MODE[1:0]		329
+0x01	CTRLB	(SRAM)	–	–	–	–	–	SRWS[2:0]			330
		(SDRAM)	SDINITDONE	–	–	–	–	SDSREN	SDMODE[1:0]		
+0x02	BASEADDRL		Chip Select Base Address Low Byte				–	–	–	–	337
+0x03	BASEADDRH		Chip Select Base Address High Byte								337

Figure 13: EBI Chip Select Register Summary

Table 33-7. Port H - alternate functions.

PORT H	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	53							
VCC	54							
PH0	55	SYNC	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$
PH1	56	SYNC	$\overline{CAS}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$
PH2	57	SYNC/ASYN	$\overline{RAS}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$
PH3	58	SYNC	$\overline{DQM}$		$\overline{ALE2}$			$\overline{ALE2}$
PH4	59	SYNC	BA0	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$
PH5	60	SYNC	BA1	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$
PH6	61	SYNC	CKE	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$
PH7	62	SYNC	CLK	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$

Notes: 1. CS0 - CS3 can optionally be moved to Port E or F
2. A16-A23 can optionally be moved to Port E or F when EBI configured in 4PORT mode. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 14: PORTH Alternate Functions

Table 33-8. Port J - alternate functions.

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	63							
VCC	64							
PJ0	65	SYNC	D0	D0	D0	D0/A0	D0/A0	D0/A0/A8
PJ1	66	SYNC	D1	D1	D1	D1/A1	D1/A1	D1/A1/A9

ATxmega128A1U/ATxmega64A1U

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
PJ2	67	SYNC/ASYN	D2	D2	D2	D2/A2	D2/A2	D2/A2/A10
PJ3	68	SYNC	D3	D3	D3	D3/A3	D3/A3	D3/A3/A11
PJ4	69	SYNC	A8	D4	D4	D4/A4	D4/A4	D4/A4/A12
PJ5	70	SYNC	A9	D5	D5	D5/A5	D5/A5	D5/A5/A13
PJ6	71	SYNC	A10	D6	D6	D6/A6	D6/A6	D6/A6/A14
PJ7	72	SYNC	A11	D7	D7	D7/A7	D7/A7	D7/A7/A15

Figure 15: PORTJ Alternate Functions

Table 33-9. Port K - alternate functions.

PORT K	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE2	LPC3 ALE1
GND	73					
VCC	74					
PK0	75	SYNC	A0	A0/A8	A0/A8/A16	A8
PK1	76	SYNC	A1	A1/A9	A1/A9/A17	A9
PK2	77	SYNC/ASYNC	A2	A2/A10	A2/A10/A18	A10
PK3	78	SYNC	A3	A3/A11	A3/A11/A19	A11
PK4	79	SYNC	A4	A4/A12	A4/A12/A20	A12
PK5	80	SYNC	A5	A5/A13	A5/A13/A21	A13
PK6	81	SYNC	A6	A6/A14	A6/A14/A22	A14
PK7	82	SYNC	A7	A7/A15	A7/A15/A23	A15

Figure 16: PORTK Alternate Functions

23.16 Register summary

23.16.1 Register description – USART

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	FERR	BUFOVF	PERR	–	RXB8	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	CLK2X	MPCM	TXB8	296
+0x05	CTRLC	CMODE[1:0]		PMODE[1:0]		SBMODE	CHSIZE[2:0]			296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

23.16.2 Register description – USART in SPI Master Mode

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	–	–	–	–	–	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	–	–	–	296
+0x05	CTRLC	CMODE[1:0]		–	–	–	UDORD	UCPHA	–	296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

Figure 17: USART Register Summaries

Table 33-3. Port C - alternate functions.

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USART0 ⁽³⁾	USART1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
GND	13										
VCC	14										
PC0	15	SYNC	OC0A	OC0ALS					SDA		

ATxmega128A1U/ATxmega64A1U

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USART0 ⁽³⁾	USART1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
PC1	16	SYNC	OC0B	OC0AHS		XCK0			SCL		
PC2	17	SYNC/ASYNC	OC0C	OC0BLS		RXD0					
PC3	18	SYNC	OC0D	OC0BHS		TXD0					
PC4	19	SYNC		OC0CLS	OC1A			SS			
PC5	20	SYNC		OC0CHS	OC1B		XCK1	MOSI			
PC6	21	SYNC		OC0DLS			RXD1	MISO		clk _{RTC}	
PC7	22	SYNC		OC0DHS			TXD1	SCK		clk _{PER}	EVOUT

- Notes:
1. Pin mapping of all TC0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 2. If TC0 is configured as TC2 all eight pins can be used for PWM output. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 3. Pin mapping of all USART0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 4. Pins MOSI and SCK for all SPI can optionally be swapped. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 5. CLKOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.
 6. EVOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.

Figure 18: PORTC Alternate Functions

Notes: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOOUT register description in the XMEGA AU Manual.

Table 33-6. Port F - alternate functions.

PORT F	PIN #	INTERRUPT	TCF0	TCF1	USARTF0	USARTF1	SPIF	TWIF
GND	43							
VCC	44							
PF0	45	SYNC	OC0A					SDA
PF1	46	SYNC	OC0B		XCK0			SCL
PF2	47	SYNC/ASYNC	OC0C		RXD0			
PF3	48	SYNC	OC0D		TXD0			
PF4	49	SYNC		OC1A			$\overline{SS}$	
PF5	50	SYNC		OC1B		XCK1	MOSI	
PF6	51	SYNC				RXD1	MISO	
PF7	52	SYNC				TXD1	SCK	

Note: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOOUT register description in the XMEGA AU Manual.

Figure 19: PORTF Alternate Functions

22.8 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRL	CLK2X	ENABLE	DORD	MASTER	MODE[1:0]		PRESCALER[1:0]		276
+0x01	INTCTRL	–	–	–	–	–	–	INTLVL[1:0]		277
+0x02	STATUS	IF	WRCOL	–	–	–	–	–	–	277
+0x03	DATA	DATA[7:0]								278

22.9 Interrupt vector summary

Table 22-4. SPI interrupt vector and its offset word address.

Offset	Source	Interrupt Description
0x00	SPI_vect	SPI interrupt vector

Figure 20: SPI Register Summary

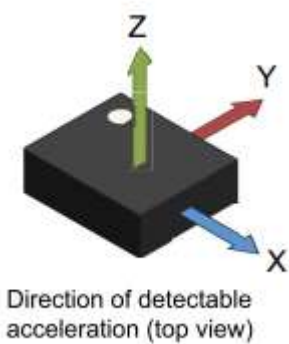


Figure 21: Axis Directions for IMU